

ASSIGNMENT:-08

NAME:-SHUBHRADEEP BISWAS

ROLL NO. :-CSB24057

SUBJECT:-ADVANCED PROGRAMMING

```
import { useState, useMemo, useEffect } from "react";
```

```
let nextId = 4;
```

```
const initialMap = new Map([
```

```
  [1, { id: 1, name: "Aria Chen", enrolledCourses: new Set(["Algorithms", "Machine Learning", "Linear Algebra"]), gpa: 3.9 }],
```

```
  [2, { id: 2, name: "Marcus Webb", enrolledCourses: new Set(["Algorithms", "Systems Programming", "Networks"]), gpa: 3.5 }],
```

```
  [3, { id: 3, name: "Priya Nair", enrolledCourses: new Set(["Machine Learning", "Statistics", "Linear Algebra"]), gpa: 3.7 }],
```

```
]);
```

```
const FIRST_NAMES =
```

```
["Aria", "Marcus", "Priya", "James", "Sofia", "Liam", "Emma", "Noah", "Olivia", "Ethan", "Ava", "Mason", "Isabe
```

```
lla", "Logan", "Mia", "Lucas", "Charlotte", "Aiden", "Amelia", "Jackson", "Harper", "Sebastian", "Evelyn", "Mat
```

```
eo", "Abigail", "Jack", "Emily", "Owen", "Elizabeth", "Theodore", "Mila", "Asher", "Ella", "Henry", "Riley", "Alexander", "Zoey", "Daniel", "Nora", "Michael", "Lily"];
```

```
const LAST_NAMES =
```

```
["Chen", "Webb", "Nair", "Smith", "Johnson", "Williams", "Brown", "Jones", "Garcia", "Miller", "Davis", "Wilson", "Taylor", "Anderson", "Thomas", "Jackson", "White", "Harris", "Martin", "Thompson", "Young", "Lee", "Walker", "Hall", "Allen", "King", "Scott", "Green", "Baker", "Adams"];
```

```
// — Complexity Panel —————
```

```

function ComplexityPanel({ allCourses }) {
  const sizes = [100, 200, 400, 600, 800, 1000];

  const rows = useMemo(() => {
    const courses = [...allCourses];
    if (!courses.length) return [];

    return sizes.map(n => { // Generate n random students
      const students =
        Array.from({ length: n }, () => {
          const num = 1 + Math.floor(Math.random() *
            Math.min(4, courses.length));
          const shuffled = [...courses].sort(() =>
            Math.random() - 0.5);

          return { name: `S${n}`, gpa: Math.random() * 2 + 2, enrolledCourses: new Set(shuffled.slice(0,
            num)) };
        });

      // Measure time to filter by a random course (repeat 50x for stable ms reading)
      const course = courses[Math.floor(Math.random() * courses.length)];
      const REPS = 50;
      const t0 = performance.now();
      for (let r = 0; r < REPS; r++) {
        students.filter(s => s.enrolledCourses.has(course));
      }

      const ms = (performance.now() - t0) / REPS;

      return { n, ms: ms < 0.01 ? "<0.01" : ms.toFixed(3) };
    });
  }, [allCourses]);

  const thStyle = {
    fontSize: 10, color: "#aaa", fontWeight: 500, padding: "6px
    8px", borderBottom: "1px solid #f0f0f0", whiteSpace: "nowrap", };

  const tdStyle = {
    fontSize: 12, color: "#555", padding:
    "6px 8px", borderBottom: "1px solid #f5f5f5",
  };

```

```

return (
  <div style={{ background: "#fff", borderRadius: 10, padding: 18, border: "1px solid #e8e8e8" }}>
    <div style={{ display: "flex", justifyContent: "space-between", alignItems: "baseline",
marginBottom: 8 }}>
      <p style={{ fontSize: 12, fontWeight: 600, color: "#555", textTransform: "uppercase",
letterSpacing: "0.06em", margin: 0 }}>
        Time Complexity
      </p>
      <span style={{ fontSize: 12, fontWeight: 700, fontFamily: "monospace", color: "#3b82f6"
}}>O(n)</span>
    </div>
    <p style={{ fontSize: 11, color: "#aaa", marginBottom: 12, lineHeight: 1.5 }}>
      Filter students by course · avg over 50 runs
    </p>

    <table style={{ width: "100%", borderCollapse: "collapse" }}>
      <thead>
        <tr>
          <th style={{ ...thStyle, textAlign: "left" }}>Students (n)</th>
          <th style={{ ...thStyle, textAlign: "right" }}>Time (ms)</th>
        </tr>
      </thead>
      <tbody>
        {rows.map(row => (
          <tr key={row.n}>
            <td style={{ ...tdStyle, textAlign: "left", color: "#111", fontWeight: 500 }}>{row.n}</td>
            <td style={{ ...tdStyle, textAlign: "right", fontFamily: "monospace", color: "#111"
}}>{row.ms}</td>
          </tr>
        ))}
      </tbody>

```

```

    </table>

  </div>

);
}

// — Main App —————

export default function App() {  const [studentsMap,
setStudentsMap] = useState(initialMap);  const [filterCourse,
setFilterCourse] = useState("");  const [form, setForm] = useState({
name: "", courses: "", gpa: "" });  const [error, setError] =
useState("");

  useEffect(() => {  document.documentElement.style.cssText =
"width:100%;height:100%;margin:0;padding:0;background:#f4f4f4;";
document.body.style.cssText =
"width:100%;minheight:100vh;margin:0;padding:0;background:#f4f4f4;";  const root
= document.getElementById("root");  if (root) root.style.cssText = "width:100%;min-
height:100vh;background:#f4f4f4;";

  }, []);

  const allStudents = useMemo(() => [...studentsMap.values()], [studentsMap]);

  const allCourses = useMemo(() =>  allStudents.reduce((acc, s) => {
s.enrolledCourses.forEach(c => acc.add(c)); return acc; }, new Set()),

  [allStudents]

);

  const displayed = useMemo(() => {  const
filtered = filterCourse

  ? allStudents.filter(s => s.enrolledCourses.has(filterCourse))

```

```

      : allStudents; return [...filtered].sort((a, b)
=> b.gpa - a.gpa);
    }, [allStudents, filterCourse]);

```

```

function addStudent() { const name = form.name.trim(); const gpa = parseFloat(form.gpa);
const courses = form.courses.split(",").map(c => c.trim()).filter(Boolean); if (!name) return
setError("Name required."); if (isNaN(gpa)) return setError("Enter a valid GPA."); if
(!courses.length) return setError("At least one course required."); setError(""); const id =
nextId++; setStudentsMap(prev => new Map([...prev, [id, { id, name, enrolledCourses: new
Set(courses), gpa }]])); setForm({ name: "", courses: "", gpa: "" });
}

```

```

function removeStudent(id) { setStudentsMap(prev => { const next = new Map([...prev]);
next.delete(id); return next; });
}

```

```

const inputStyle = { width: "100%", padding: "8px
10px", fontSize: 13, background: "#f9f9f9", border:
"1px solid #e0e0e0", borderRadius: 6, outline: "none",
color: "#111",

```

```

marginBottom: 8, boxSizing: "border-box",
};

```

```

return ( <div style={{ fontFamily: "'Inter', system-ui, sans-
serif", background: "#f4f4f4", minHeight: "100vh", width:
"100vw", boxSizing: "border-box", padding: 32, position:
"absolute", top: 0, left: 0,
}}>

```

```

<style>{`
@import
url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600&display=swap');

```

```
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }    html, body, #root
{ width: 100% !important; min-height: 100vh !important; background: #f4f4f4 !important; }

`</style>
```

```
<div style={{ marginBottom: 28 }}>

  <h1 style={{ fontSize: 20, fontWeight: 600, color: "#111", margin: 0 }}>Course Enrollment</h1>

  <p style={{ fontSize: 13, color: "#888", marginTop: 4 }}>

    {allStudents.length} students · {allCourses.size} unique courses

  </p>

</div>
```

```
<div style={{ display: "grid", gridTemplateColumns: "280px 1fr", gap: 24, alignItems: "start" }}>
```

```
<div style={{ display: "flex", flexDirection: "column", gap: 16 }}>
```

```
  {/* Add student */}
```

```
  <div style={{ background: "#fff", borderRadius: 10, padding: 18, border: "1px solid #e8e8e8"
}}>
```

```
    <p style={{ fontSize: 12, fontWeight: 600, color: "#555", textTransform: "uppercase",
letterSpacing: "0.06em", margin: "0 0 12px" }}>Add Student</p>
```

```
    <input style={inputStyle} placeholder="Name" value={form.name} onChange={e => setForm(p
=> ({ ...p, name: e.target.value })))} />
```

```
    <input style={inputStyle} placeholder="GPA (any number)" type="number" value={form.gpa}
onChange={e => setForm(p => ({ ...p, gpa: e.target.value })))} />
```

```
    <input style={inputStyle} placeholder="Courses, comma separated" value={form.courses}
onChange={e => setForm(p => ({ ...p, courses: e.target.value })))} />
```

```
    {error && <p style={{ fontSize: 12, color: "#e55", marginBottom: 8 }}>{error}</p>}
```

```
    <button onClick={addStudent} style={{ width: "100%", background: "#111", color: "#fff",
border: "none", borderRadius: 6, padding: "9px 0", fontSize: 13, fontWeight: 500, cursor: "pointer"
}}>
```

```
      Add Student
```

```
</button>
```

```
</div>
```

```

    {/* Filter */}

    <div style={{ background: "#fff", borderRadius: 10, padding: 18, border: "1px solid #e8e8e8" }}>

      <p style={{ fontSize: 12, fontWeight: 600, color: "#555", textTransform: "uppercase",
letterSpacing: "0.06em", margin: "0 0 10px" }}>Filter by Course</p>

      <select value={filterCourse} onChange={e => setFilterCourse(e.target.value)} style={{
...inputStyle, marginBottom: 0, cursor: "pointer" }}>

        <option value="">All courses</option>

        {[...allCourses].sort().map(c => <option key={c} value={c}>{c}</option>)}

      </select>

      {filterCourse && (

        <button onClick={() => setFilterCourse("")} style={{ marginTop: 8, fontSize: 12, color:
"#999", background: "none", border: "none", cursor: "pointer", padding: 0 }}>

          Clear filter ×

        </button>

      )}

    </div>

    {/* All courses */}

    <div style={{ background: "#fff", borderRadius: 10, padding: 18, border: "1px solid #e8e8e8" }}>

      <p style={{ fontSize: 12, fontWeight: 600, color: "#555", textTransform: "uppercase",
letterSpacing: "0.06em", margin: "0 0 10px" }}>All Courses</p>

      <div style={{ display: "flex", flexWrap: "wrap", gap: 6 }}>

        {[...allCourses].sort().map(c => (

          <span key={c} style={{ fontSize: 12, background: "#f0f0f0", color: "#444", padding: "3px
8px", borderRadius: 4 }}>{c}</span>

        ))}

      </div>

    </div>

    <ComplexityPanel allCourses={allCourses} />

  </div>

  {/* Student rows */}

  <div>

```

```

{filterCourse && (
  <p style={{ fontSize: 13, color: "#555", margin: "0 0 10px" }}>
    {displayed.length} student{displayed.length !== 1 ? "s" : ""} in <strong>{filterCourse}</strong>
  </p>
)}

{displayed.length === 0 && (
  <div style={{ background: "#fff", borderRadius: 10, padding: 40, textAlign: "center", color:
"#bbb", border: "1px solid #e8e8e8", fontSize: 14 }}>      No students found.
  </div>
)}

<div style={{ display: "flex", flexDirection: "column", gap: 8 }}>
  {displayed.map((student, idx) => (
    <div key={student.id} style={{ background: "#fff", borderRadius: 10, padding: "14px 18px",
border: "1px solid #e8e8e8", display: "flex", alignItems: "center", gap: 14 }}>

      <span style={{ fontSize: 12, color: "#ccc", width: 22, textAlign: "center", fontWeight: 500,
flexShrink: 0 }}>#{idx + 1}</span>

      <div style={{ width: 34, height: 34, borderRadius: "50%", background: "#111", color: "#fff",
display: "flex", alignItems: "center", justifyContent: "center", fontSize: 13, fontWeight: 600, flexShrink:
0 }}>

        {student.name.charAt(0)}

      </div>

      <div style={{ flex: 1, minWidth: 0 }}>

        <div style={{ fontSize: 14, fontWeight: 600, color: "#111" }}>{student.name}</div>

        <div style={{ display: "flex", flexWrap: "wrap", gap: 4, marginTop: 5 }}>

          {[...student.enrolledCourses].map(c => (

            <span key={c} style={{ fontSize: 11, background: "#f0f0f0", color: "#555", padding:
"2px 7px", borderRadius: 4 }}>{c}</span>

          ))}

        </div>
      </div>
    )
  )}

```



```
</div>
```

```
<div style={{ textAlign: "right", flexShrink: 0 }}>
```

```
  <div style={{ fontSize: 15, fontWeight: 600, color: "#111" }}>{student.gpa.toFixed(2)}</div>
```

```
  <div style={{ fontSize: 11, color: "#bbb" }}>GPA</div>
```

```
</div>
```

```
<button
```

```
  onClick={() => removeStudent(student.id)}          style={{ background: "none",  
border: "1px solid #e8e8e8", borderRadius: 6, padding: "5px 10px", fontSize: 12, color: "#aaa",  
cursor: "pointer", flexShrink: 0 }}
```

```
>
```

```
  Remove
```

```
</button>
```

```
</div>
```

```
  )}
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
);
```

```
}
```